

Notas de Apresentação – Defesa Oral (1 hora)

Avaliação 03: Solução Big Data · MAE016.15912 · UFRJ/NCE · Prof. Milton Ramos Ramirez

Grupo: Izabela Lima da Silva (DRE 124156557) · Caio Meirelles (DRE 122071557) · 25 slides

FORMATO

1 hora total → ~45 min de apresentação + ~15 min de perguntas do professor. Ritmo médio ~2 min/slide (blocos técnicos da Parte A podem ir mais fundo). O critério da avaliação é **explicar e justificar** cada escolha — sempre conectando à **teoria (referência citada)** e ao **valor de negócio**.

DIVISÃO DA DEFESA (IZABELA / CAIO)

- **COMUM** slides **1–9** (abertura → arquitetura) e **20–25** (casos de uso, custos, alternativas, referências, conclusões).
- **PARTE A** slides **10–15** — borda, Kafka, conectividade, Lambda×Kappa, Delta Lake, processamento.
- **PARTE B** slides **16–19** — persistência poliglota, dashboard, cobrança+emergências, concierge.
- **PONTE A↔B** slide **18** (cobrança e emergências) — bom ponto de troca de bastão.
- Sugestão: quem defende a Parte A abre (1–9); quem defende a Parte B fecha (20–25).

Bloco	Slides	Tempo
Abertura + contexto + valor	1–4	~7 min
Requisitos + metodologia + fundamentos	5–7	~8 min
DW/extensão + arquitetura	8–9	~5 min
Parte A (borda → processamento)	10–15	~13 min
Parte B (persistência → concierge)	16–19	~8 min
Casos de uso + rastreabilidade	20–21	~4 min
Custos + alternativas + referências + conclusões	22–25	~4 min

1 Capa

≈40 s

- › Apresentar o grupo (Izabela + Caio, com DRE), a disciplina e o tema.
- › Gancho: *"Cada veículo da nova frota é um sensor sobre rodas. Vamos mostrar uma arquitetura que transforma esse dilúvio de dados em decisão de negócio em tempo real — fundamentada na literatura e voltada a valor mensurável para o consórcio."*

2 Índice

≈30 s

- › 18 seções em 25 slides. Sinalizar que a defesa **cobre os dois trabalhos** (DW da Aval. 02 + Big Data) e as **etiquetas Parte A/B** mostram quem defende cada bloco.

3 Contexto e problema

≈1 min 30

- › Cenário: consórcio de 6 locadoras → veículos autônomos que se deslocam sozinhos.
- › Três vetores: fonte de dados (veículo instrumentado), escala (frota simultânea), ambição (autonomia + IA).
- › Fechar com a tese: **"o DW sozinho não basta"** — é preciso Big Data.

4 Valor para o cliente SLIDE DE NEGÓCIO

≈2 min

- › Aqui você "vende" a solução. Percorrer os 6 ganhos e nomear o indicador:
 - Utilização (Markov + rotas) · Receita (cobrança dinâmica) · Custo (sem balcão) · Sinistros (resposta/dossiê) · Downtime (manutenção preditiva) · Experiência (concierge).
- › Frase-âncora: *"cada decisão técnica a seguir existe para mover um destes indicadores — a arquitetura é meio, o valor é o fim."*

5 Objetivos e requisitos

≈2 min

- › **Funcionais:** dashboard, reserva, cobrança automática, emergências + extra (concierge).
- › **Não funcionais (5 desafios):** banda finita, alta concorrência, ordenação estrita, resiliência, ACID. Dizer: *"estes cinco requisitos guiam TODAS as decisões seguintes."*

6 Metodologia

≈2 min

- › 6 etapas: requisitos → **revisão de 21 referências primárias** (papers seminais, mapeados às 4 partes da ementa) → decisão fundamentada → seleção por padrão de acesso → **prova de conceito executável** (docker-compose + DDL validado) → continuidade com o DW.
- › Enfatizar o rigor: *"não escolhemos por moda; cada camada tem lastro teórico citado e foi validada em código."*

7

Fundamentos teóricos

O PROFESSOR VAI COBRAR

≈3 min

- Percorrer a tabela pelas 4 partes da ementa, 1 frase por linha, citando o autor:
 - I — falha é a norma (Ghemawat 2003) e localidade (Dean & Ghemawat 2004); ordem/TrueTime (Corbett 2012).
 - II — OLAP separado do OLTP e CUBE (Chaudhuri & Dayal 1997; Gray 1997); Lakehouse (Armbrust 2021).
 - III — log particionado (Kreps 2011), watermarks (Carbone 2015), exactly-once (Zaharia 2013); RAG (Lewis 2020).
 - IV — CAP (Gilbert & Lynch 2002), Dynamo/Cassandra (DeCandia 2007; Lakshman 2009).
- Deixar claro: **cada princípio ancora uma decisão nossa** (coluna "onde aplicamos").

8

Do DW ao Big Data + extensão

COBRE A AVAL. 02

≈2 min 30

- **Esquerda (nosso DW real):** esquema estrela PostgreSQL; fatos Reserva/Locação/Movimentação (fact constellation); dims conformadas; 4 OLTP; Markov. OLAP separado do OLTP (Chaudhuri & Dayal 1997).
- **Direita (a EXTENSÃO — novidade da Aval. 03):** 4 novos fatos (Telemetria, Cobrança, Sinistro, Manutenção) + novas dims (Tempo_Detalhe minuto, Sensor SCD-2, TipoEvento, FaixaCondução), carregados por ELT do Silver. **DDL validado** rodando 01+06 num PostgreSQL real.
- **Reconciliação com a Parte 2:** a *Fato_Cobranca* dá continuidade ao **Pagamento** (valor) do OLTP da Parte I — que foi esboçado como *Fato_Pagamento* —, agora materializado na Gold e enriquecido com ajustes dinâmicos. Não é "lacuna", é evolução.
- **Mensagem:** não substituímos o DW — ele **vira a camada Gold** do Lakehouse (Armbrust 2021). Continuidade, não retrabalho.

9

Arquitetura de referência

SLIDE CENTRAL

≈2 min 30

- Percorrer o pipeline: Borda → Kafka → Spark/Flink → Delta (Bronze/Silver/Gold) → NoSQL → Serving.
- Explicar os **3 caminhos** (hot/warm/cold). É o mapa; volte a ele se a banca se perder.

PARTE A

Infraestrutura e fluxo

slides 10-15

10

Borda / Edge

≈2 min 15

- Fundamento: **localidade** — o *combiner* do MapReduce aplicado à borda (Dean & Ghemawat 2004).
- **Dois fluxos:** rotina (Avro/MQTT a cada 1 s) vs. crítico (colisão detectada na borda, reação sub-segundo, ring buffer ~30 s).
- **Valor:** corta o maior custo recorrente (egress); a decisão crítica não espera a nuvem; store-and-forward = zero perda.

11 Ingestão / Kafka

≈2 min 15

- › Kafka = **log distribuído particionado** (Kreps 2011): pull por offset, retenção, buffer durável sob banda finita.
- › **Tópicos isolados** (`telemetry.raw` vs `alerts.critical`) — o alarme nunca fica preso. **Partição por** `vehicle_id` → ordem por veículo sem coordenação global. Replicação/ISR (Ghemawat 2003). At-least-once + dedup por chave. Avro + Schema Registry.

12 Conectividade e ordenação COBRE A3

≈2 min

- › Zonas de sombra (túneis) → dados atrasados/fora de ordem.
- › Três camadas: **borda** (Avro + store-and-forward), **Kafka** (ordem por `vehicle_id`), **Flink** (event-time + **watermarks** reordenam pelo timestamp do veículo).
- › Analogia forte: o rigor de ordem que o **Spanner** obtém com TrueTime (Corbett 2012), o Flink emula com watermarks (Carbone 2015).

13 Lambda vs. Kappa DECISÃO COBRADA

≈2 min

- › Ler a tabela por critérios; reconhecer a vantagem do Lambda (batch nativo).
- › **Decisão**: o stream durável do Kafka unifica os caminhos e elimina a dupla implementação (Fragkoulis 2020) → **Kappa**, com o Lakehouse cobrindo o lote pesado.

14 Lakehouse / Delta Lake

≈2 min

- › O object store é key-value sem atomicidade cross-key → o **log de transações do Delta** traz ACID, data skipping e time travel (Armbrust 2020).
- › Medallion Bronze → Silver → **Gold (o DW estendido)**. Motor: Spark SQL + Catalyst (Armbrust 2015). É a resposta ao "ACID entre batch e streaming".

15 Processamento: três regimes

≈2 min 30

- › **Não é redundância — é adequação de SLA:**
 - **Spark batch** — RDD/linhagem, reuso em memória até 20× (Zaharia 2012); Markov; generaliza o MapReduce.
 - **Spark Streaming** — hot path, micro-batches com **exactly-once** idempotente (Zaharia 2013), evita cobrar em dobro.
 - **Flink** — event-time + watermarks + ABS/Chandy-Lamport (Carbone 2015); rotas e score.

16 Persistência poliglota

≈2 min 15

- › Raiz: **CAP** (Gilbert & Lynch 2002) — segmentar e dar a cada subserviço seu trade-off (Cattell 2011).
- › Cassandra (AP, telemetria, LSM + clustering por ts materializa a ordem — Lakshman 2009 / DeCandia 2007) · MongoDB (cadastrais/dossiês) · Redis (cache sub-ms) · PostgreSQL/Delta (CP, cobrança/Gold).
- › Frase: *"telemetria é AP, cobrança é CP — o mesmo trade-off carrinho vs. checkout do Dynamo."*

17 Dashboard / Streamlit

≈2 min

- › OLAP sobre a **Gold**, nunca sobre os OLTP (Chaudhuri & Dayal 1997); ao vivo via Redis pub/sub.
- › Cubo frota × tempo × pátio × empresa (Gray 1997). Percorrer os painéis (frota, financeiro, emergências, viagens, oficina) + **trânsito, meteorologia e eventos/alertas da cidade** — mapeiam 1:1 com o enunciado.
- › Frisar: o mesmo tempo real deixa a **reserva mais objetiva** (disponibilidade e ETA reais no app) — a "melhoria da reserva" que o enunciado pede.

18 Cobrança e emergências

PONTE A↔B

≈2 min 15

- › **Cobrança**: gatilho na devolução → valor final = base ± ajustes (km, tempo, consumo, infrações, score do Flink). Exactly-once evita duplicidade; auditável por time travel. **Valor: ↑ receita, ↓ contestações.**
- › **Emergências**: borda detecta → `alerts.critical` → Spark orquestra → resposta (guincho, seguradora, substituto) + **dossiê citável**. **Valor: ↓ tempo de resposta e de regulação.**

19 Concierge de IA

≈2 min

- › Backend de voz: PLN + agente + **RAG** sobre **banco vetorial** (Lewis 2020; Guo 2022) — sem chave de LLM paga.
- › RAG dá **proveniência auditável** e reduz alucinação — a mesma base sustenta os dossiês de emergência. Usar o exemplo do almoço + exposição de arte. Mencionar privacidade (consentimento, PII, wake-word).

20 Casos de uso reais

A MODELAGEM EM AÇÃO

≈2 min

- › Três cenários: **Colisão em túnel** (crítico/resposta rápida), **Devolução com condução agressiva** (usual/financeiro), **Pico no aeroporto + pane + concierge** (analítico/IA).
- › Para cada um: diga o cenário, o fluxo pelo pipeline e **quais entidades "acendem"**. Enfatize que três delas **justificam a extensão do modelo** (Dim_Tempo_Detalhe, Fato_Cobranca, Fato_Manutencao/Dim_Sensor).

21 Rastreabilidade das entidades

IMPRESSIONA O PROFESSOR

≈1 min 30

- › A matriz mostra **onde cada fato/dimensão aparece** nos três casos (crítico/usual/analítico) e marca com **+** o que é da extensão.
- › Frase: *"consequimos apontar cada entidade modelada numa situação real — usual ou crítica."* Convide o professor a escolher uma entidade da coluna e você explica em qual caso ela vive.

22 Custos & prova de conceito

≈1 min 15

- › Três alavancas: banda (borda), compute (autoscaling + spot), storage (tiering + TTL).
- › **Sistema executável**: docker-compose validado, **115 testes unitários** verdes, DDL do DW estendido validado. Free tiers para experimentar.

23 Alternativas descartadas

≈1 min

- › Responde ao critério "por que NÃO outras soluções". Uma frase por linha (só DW, só batch, Lambda, banco único, vídeo bruto, Hadoop MapReduce clássico).

24 Referências

≈45 s

- › Sinalizar que **toda decisão tem lastro**: 21 referências primárias mapeadas à ementa. Não ler tudo — apontar que a bibliografia está no repositório e na proposta.

25 Conclusões

≈1 min

- › Recapitular: fundamentada → arquitetura (borda/Kafka/Kappa+Delta/3 regimes/poliglota) → **DW estendido (Gold)** → serving+IA → **valor ao cliente** → executável.
- › Fechar: arquitetura resiliente, escalável, em tempo real, teoricamente fundamentada e que move os indicadores de negócio. Abrir para perguntas.

Por que Kappa e não Lambda?

Problema é streaming-first; Lambda mantém dois códigos e arrisca divergência (Fragkoulis 2020). O stream durável do Kafka unifica; o lote pesado é coberto pelo Lakehouse (Delta), então não perdemos reproprocessamento.

Como garantem a ordenação estrita por veículo e timestamp?

Partição do Kafka por `vehicle_id` (ordem estrita na partição — Kreps 2011) + processamento por **event-time com watermarks** no Flink (Carbone 2015); o timestamp é carimbado na borda. Mesmo objetivo da consistência externa do Spanner/TrueTime (Corbett 2012).

Exactly-once é real?

Na ingestão, **at-least-once** + chave de idempotência (`vehicle_id+timestamp`) para dedup end-to-end; **exactly-once** de fato nas etapas internas (D-Streams idempotentes — Zaharia 2013; ABS no Flink — Carbone 2015).

Como garantem "sem perda de pacotes"?

Kafka é log durável com replay por offset; a borda faz store-and-forward na desconexão; e o modelo *always-writeable* do Dynamo (DeCandia 2007) inspira a camada de telemetria (Cassandra).

Diferença entre os três motores?

Vazão histórica (Spark batch), orquestração near-real-time da emergência (Spark Streaming, segundos), latência mínima com janelas/estado (Flink, ms). A reação sub-segundo à colisão é **embarcada**, não na nuvem.

DATA WAREHOUSE & LAKEHOUSE (PARTE II)

ETL ou ELT?

ELT: carregamos o bruto no Lakehouse (Bronze) e transformamos com o poder do cluster (Silver→Gold). É a modernização que a ementa pede sobre o ETL clássico.

Como o DW da Aval. 02 se conecta?

Vira a **camada Gold** do Lakehouse (Armbrust 2021), estendida com 4 novos fatos e novas dimensões carregados por ELT do Silver — continuidade, não descarte.

O que é o Lakehouse e por que Delta?

Unifica DW e data lake numa arquitetura só, evitando *staleness* e TCO da arquitetura de dois níveis; o Delta traz ACID/time travel ao object store via log de transações (Armbrust 2020, 2021).

Onde aparece o OLAP/CUBE?

No dashboard: roll-up frota→empresa→pátio→veículo e o cubo N-dimensional com `ALL` (Gray 1997), sempre sobre a Gold, nunca sobre o OLTP (Chaudhuri & Dayal 1997).

Orquestração dos pipelines?

No perfil `full`, um DAG no **Airflow** orquestra o ELT Silver→Gold (a ementa cita Airflow/Prefect/Dagster).

NOSQL, NEWSQL & VETORIAIS (PARTE IV)

Por que Cassandra e não só MongoDB?

Cassandra é *write-optimized* (LSM) e faz *clustering* por timestamp dentro da partição `vehicle_id`, materializando a ordem (Lakshman 2009) — ideal para telemetria de altíssima escrita; Mongo é para cadastrais/dossiês semiestruturados.

Onde cada banco cai no CAP?

Telemetria = **AP** (disponível, consistência eventual); cobrança/dossiê = **CP** (consistência forte, PostgreSQL/Delta). Segmentamos por trade-off (Gilbert & Lynch 2002; Cattell 2011).

O que é NewSQL e onde entra?

SQL + ACID que escala horizontalmente (Spanner — Corbett 2012). Aqui o PostgreSQL é o polo consistente da cobrança e da modelagem dimensional; snapshot no passado espelha o time travel do Delta.

Banco vetorial — qual e por quê?

Para a memória do concierge (embeddings de POIs), busca por similaridade (Manu — Guo 2022; pgvector/Milvus/Weaviate na ementa). Liga a Parte III (RAG) à Parte IV.

IA (PARTE III)

O que é RAG e por que usar?

Combina memória paramétrica (gerador) e não-paramétrica (índice denso); reduz alucinação, dá **proveniência** e permite trocar o índice sem re-treino (Lewis 2020). Fundamenta o concierge (sem LLM paga) e os dossiês citáveis.

Agentes de IA / MLOps?

No perfil `full`: pipeline MLOps (sklearn + MLflow) treina o score de condução/manutenção preditiva; o agente do concierge usa function calling. Grafo (Neo4j) para roteirização/raciocínio.

FUNDAMENTOS & VALIDAÇÃO

MapReduce ainda importa?

Como **paradigma**: restringir a map/reduce é o que dá paralelismo e tolerância por re-execução (Dean & Ghemawat 2004). O Spark o **generaliza** (DAG, in-memory, linhagem — Zaharia 2012); por isso descartamos o Hadoop MapReduce clássico (I/O em disco).

Como validaram sem rodar tudo?

`docker compose config` valida os perfis core e full; **115 testes unitários** (combiner, cobrança exactly-once, Markov=1.0, SCD-2, RAG); o DDL 01+06 do DW roda limpo num PostgreSQL real. O `docker compose up` completo roda na máquina de 16 GB.

Custos de nuvem?

Maior custo é banda/egress → filtramos na borda; compute com autoscaling + spot; storage com tiering (quente Redis/Cassandra vs. frio Delta) e TTL.

Privacidade / LGPD?

Consentimento para agenda, tratamento de PII, wake-word on-device no concierge; no `full`, controles de governança/LGPD e time travel do Delta para auditoria.

Como o sistema lida com casos limítrofes?

Quatro garantias combinadas: **durabilidade** (store-and-forward na borda + log Kafka/Delta → zero perda em túnel/queda), **ordem** (partição por veículo + watermarks reordenam atrasados; eventos muito tardios reconciliam no lote), **idempotência** (dedup por chave; cobrança exactly-once — nunca em dobro) e **reconciliação** (time travel para evento tardio pós-fatura). Sob partição de rede: telemetria AP segue aceitando escrita; cobrança/dossiê CP aguardam consistência. Sensor/firmware inválido → Schema Registry + dead-letter.

O DW da Aval. 02 não tinha finanças?

Tinha no conceito: o **Pagamento** (valor) existia no OLTP da Parte I e foi esboçado como Fato_Pagamento. O esquema estrela entregue consolidou 3 fatos de contagem/duração; a **Fato_Cobranca** traz esse pagamento para a Gold e o enriquece com km/consumo/infração/score — **continuidade**, não lacuna.